

EcoBuck Smart Compost Monitoring System

REST API Design Specification (OpenAPI / Swagger Compliance)

Author: Toqeer Ahmed

Role: Principal IoT Systems & Backend Architect

Date: July 8, 2026

Status: PROPOSED (API Review Stage)

1. Architectural Overview & Security Scheme

This specification outlines the REST APIs connecting: 1. **The EcoBuck IoT Device (Firmware) to the Backend** 2. **The Mobile Application to the Backend**

1.1 Authentication Protocols

- **IoT Device Auth:** Devices authenticate using a static cryptographically signed API key transmitted via the `X-Device-Token` HTTP header.
 - **Mobile Client Auth:** The mobile app authenticates using standard JSON Web Tokens (JWT) conforming to OAuth2 Bearer standards, transmitted in the `Authorization: Bearer <token>` header.
-

2. Firmware-to-Backend Interface

2.1 Post Telemetry Payload

- **Purpose:** Ingests sensor readings, FSM state, and battery status from field-deployed ESP32 devices.
 - **HTTP Method:** `POST`
 - **Route:** `/api/v1/telemetry`
 - **Authentication:** Device API Token
- Headers:**

- **Content-Type:** application/json
- **X-Device-Token:** ecobuck_device_api_token_hash_here
- **Request Body (JSON):** `json { "device_id": "ecobuck-dev-001", "firmware_version": "0.1.0", "schema_version": 1, "timestamp": 1783515600, "readings": { "temperature_c": { "raw": 44.5, "filtered": 44.7, "trend_avg": 44.2, "quality": "good", "error": false }, "humidity_pct": { "raw": 55.2, "quality": "good", "error": false } }, "status": "active_composting", "quality_flag": "good", "cycle_age_days": 1.25, "battery_pct": 87 }`
- **Response Body (202 Accepted - JSON):** `json { "status": "success", "message": "Telemetry accepted", "timestamp": 1783515605 }`

Error Codes:

- **400 Bad Request:** Payload malformed or schema version unsupported.
- **401 Unauthorized:** X-Device-Token missing or invalid.
- **429 Too Many Requests:** Rate limit exceeded (Max 2 requests/minute).

Validation Rules:

- **device_id** must match a registered device in the database.
- **timestamp** must be within ± 10 minutes of the backend system clock.
- **battery_pct** must be an integer between 0 and 100.

Possible Exceptions:

- **ClockSkewException:** Fired if ESP32 RTC module drifts out of bounds. Overrides payload timestamp with server time.
 - **DeviceSuspendedException:** Fired if the device registry flags the hardware as banned or deactivated.
 - **Swagger Documentation (YAML):** `yaml /api/v1/telemetry: post: summary: Post Telemetry Payload description: Ingests raw and filtered telemetry from the ESP32 firmware. security: - DeviceTokenAuth: [] parameters: - in: header name: X-Device-Token required: true schema: type: string requestBody: required: true content: application/json: schema: type: object required: [device_id, firmware_version, schema_version, timestamp, readings, status, quality_flag, cycle_age_days] properties: device_id: type: string firmware_version: type: string schema_version: type: integer timestamp: type: integer readings: type: object status: type: string quality_flag: type: string cycle_age_days: type: number battery_pct: type: integer responses: '202': description: Telemetry accepted for processing. '400': description: Malformed payload. '401': description: Invalid device token.`
-

2.2 Get Device Configuration

- **Purpose:** Permits the ESP32 device to check for calibration offsets, sleep timing, or threshold changes provisioned via the mobile app.
- **HTTP Method:** GET
- **Route:** /api/v1/devices/config
- **Authentication:** Device API Token
- **Headers:**
 - X-Device-Token: ecobuck_device_api_token_hash_here
- **Request Body:** None
- **Response Body (200 OK - JSON):**

```
json { "device_id": "ecobuck-dev-001", "temp_offset_c": -0.5, "humidity_offset_pct": 2.0, "read_interval_seconds": 30, "summary_interval_seconds": 300, "temp_thresholds": { "ambient_temp_c": 25.0, "active_delta_c": 10.0, "max_safety_c": 65.0 }, "humidity_thresholds": { "min_pct": 20.0, "max_pct": 75.0 } }
```
- **Error Codes:**
 - 401 Unauthorized: Invalid token.
 - 404 Not Found: Device ID not registered.
- **Validation Rules:** None (Read-only lookup).
- **Possible Exceptions:**
 - ConfigFetchException: Triggered if database connectivity fails while retrieving the parameters.
- **Swagger Documentation (YAML):**

```
yaml /api/v1/devices/config: get: summary: Get Device Configuration description: Returns operational config and calibration values for the requesting device. security: - DeviceTokenAuth: [] responses: '200': description: Configuration details retrieved. content: application/json: schema: type: object properties: device_id: type: string temp_offset_c: type: number humidity_offset_pct: type: number read_interval_seconds: type: integer summary_interval_seconds: type: integer '401': description: Unauthorized device.
```

3. Backend-to-Mobile App Interface

3.1 User Registration

- **Purpose:** Registers a new user account on the platform.

- **HTTP Method:** `POST`
 - **Route:** `/api/v1/auth/register`
 - **Authentication:** None
- Headers:**
- `Content-Type: application/json`
- **Request Body (JSON):** `json { "email": "user@example.com", "password": "StrongPassword123!", "display_name": "Toqeer Ahmed" }`
 - **Response Body (201 Created - JSON):** `json { "user_id": "user-uuid-1111", "email": "user@example.com", "created_at": "2026-07-08T08:00:00Z" }`

Error Codes:

- `400 Bad Request`: Email structure invalid or password complexity requirements failed.
- `409 Conflict`: Email already exists.

Validation Rules:

- `email` must conform to RFC 5322 format.
- `password` must contain at least 8 characters, 1 uppercase letter, 1 number, and 1 special symbol.

Possible Exceptions:

- `WeakPasswordException`
 - `DuplicateEmailException`
- **Swagger Documentation (YAML):** `yaml /api/v1/auth/register: post: summary: Register User description: Creates a new user profile on the platform. requestBody: required: true content: application/json: schema: type: object required: [email, password, display_name] properties: email: type: string password: type: string display_name: type: string responses: '201': description: User created successfully. '400': description: Invalid input validations. '409': description: User already exists.`

3.2 User Login (Token Exchange)

- **Purpose:** Validates credentials and returns a secure JWT access token for mobile requests.
 - **HTTP Method:** `POST`
 - **Route:** `/api/v1/auth/login`
 - **Authentication:** None
- Headers:**

- **Content-Type:** application/json
- **Request Body (JSON):** `json { "email": "user@example.com", "password": "StrongPassword123!" }`
- **Response Body (200 OK - JSON):** `json { "access_token": "jwt_token_string_here", "token_type": "Bearer", "expires_in": 3600, "refresh_token": "refresh_token_string_here" }`

Error Codes:

- **401 Unauthorized:** Invalid password or username.

Validation Rules:

- Both fields are required.

Possible Exceptions:

- **AccountLockedException:** Fired after 5 consecutive failed login attempts (locks account for 15 minutes).
- **Swagger Documentation (YAML):** `yaml /api/v1/auth/login: post: summary: User Login description: Authenticates user credentials and yields a bearer JWT. requestBody: required: true content: application/json: schema: type: object required: [email, password] properties: email: type: string password: type: string responses: '200': description: Authenticated successfully. '401': description: Credentials invalid.`

3.3 List Owned Devices

- **Purpose:** Fetches metadata of all EcoBuck units registered to the logged-in user.
- **HTTP Method:** GET
- **Route:** /api/v1/devices
- **Authentication:** OAuth2 Bearer (JWT)

Headers:

- **Authorization:** Bearer <JWT>

- **Request Body:** None
- **Response Body (200 OK - JSON):** `json [ { "device_id": "ecobuck-dev-001", "label": "Backyard Bin", "status": "active_composting", "firmware_version": "0.1.0", "last_seen": "2026-07-08T08:20:00Z", "battery_pct": 87 } ]`

Error Codes:

- **401 Unauthorized:** JWT invalid or expired.
 - **Validation Rules:** None.
Possible Exceptions:
 - `TokenExpiredException`
 - **Swagger Documentation (YAML):**

```
yaml /api/v1/devices: get: summary: List Devices
description: Retrieves metadata for all devices linked to the authenticated user.
security: - BearerAuth: [] responses: '200': description: List of devices. '401':
description: Token missing or expired.
```
-

3.4 Claim Device

- **Purpose:** Pairs a physical device (using its hardware ID) to the user's account.
- **HTTP Method:** `POST`
- **Route:** `/api/v1/devices/claim`
- **Authentication:** OAuth2 Bearer (JWT)
Headers:
 - `Authorization: Bearer <JWT>`
 - `Content-Type: application/json`
- **Request Body (JSON):**

```
json { "hardware_id": "ecobuck-dev-001", "label": "Balcony
Composter" }
```
- **Response Body (200 OK - JSON):**

```
json { "status": "claimed", "device_id":
"ecobuck-dev-001", "label": "Balcony Composter", "assigned_at": "2026-07-08T08:24:50Z" }
```
- **Error Codes:**
 - **400 Bad Request:** `hardware_id` not found in registry (device must be pre-provisioned at factory).
 - **409 Conflict:** Device is already claimed by another user.
- **Validation Rules:**
 - `hardware_id` must be non-empty.
 - `label` must be under 50 alphanumeric characters.
- **Possible Exceptions:**
 - `DeviceAlreadyClaimedException`
 - `InvalidHardwareIdentifierException`

- **Swagger Documentation (YAML):** `yaml /api/v1/devices/claim: post: summary: Claim Device description: Associates an unowned hardware ID with the user's account. security: - BearerAuth: [] requestBody: required: true content: application/json: schema: type: object required: [hardware_id, label] properties: hardware_id: type: string label: type: string responses: '200': description: Claim successful. '400': description: Hardware ID unregistered. '409': description: Device owned by another user.`

3.5 Get Latest Status (Dashboard Cache)

- **Purpose:** Fetches the most recent sensor values and status for a single device, bypassing historical log scans to optimize database reads.

- **HTTP Method:** GET

- **Route:** `/api/v1/devices/{device_id}/latest`

- **Authentication:** OAuth2 Bearer (JWT)

Headers:

- `Authorization: Bearer <JWT>`

- **Request Body:** None

- **Response Body (200 OK - JSON):** `json { "device_id": "ecobuck-dev-001", "label": "Backyard Bin", "status": "ready_candidate", "last_seen": "2026-07-08T08:00:00Z", "battery_pct": 87, "current_conditions": { "temperature_c": 27.2, "humidity_pct": 45.1, "quality_flag": "good", "cycle_age_days": 14.52 } }`

Error Codes:

- `403 Forbidden:` User does not own this device.
- `404 Not Found:` Device not registered.

- **Validation Rules:** Path parameter `device_id` must match an active device.

Possible Exceptions:

- `DeviceAccessDeniedException`

- **Swagger Documentation (YAML):** `yaml /api/v1/devices/{device_id}/latest: get: summary: Get Latest Status description: Reads cached latest state from `/latest/status` document. security: - BearerAuth: [] parameters: - in: path name: device_id required: true schema: type: string responses: '200': description: Cached state details. '403': description: Access denied to this device.`
-

3.6 Get Historical Telemetry (Charts)

- **Purpose:** Fetches time-series telemetry data points for rendering charts. Supports date range boundaries and resolution step intervals to optimize performance.

- **HTTP Method:** GET

- **Route:** /api/v1/devices/{device_id}/history

- **Authentication:** OAuth2 Bearer (JWT)

Headers:

- Authorization: Bearer <JWT>

Query Parameters:

- start_time (Unix timestamp, required)
- end_time (Unix timestamp, required)
- resolution (String: raw, hourly, daily. Default: hourly)

- **Request Body:** None

- **Response Body (200 OK - JSON):** json { "device_id": "ecobuck-dev-001", "resolution": "hourly", "points_count": 2, "data": [{ "timestamp": 1783512000, "temperature_c": 44.2, "humidity_pct": 54.8 }, { "timestamp": 1783515600, "temperature_c": 44.7, "humidity_pct": 55.2 }] }

Error Codes:

- 400 Bad Request: Date range parameters are invalid or start time exceeds end time.
- 403 Forbidden: User does not own this device.

Validation Rules:

- Maximum range window cannot exceed 90 days.

Possible Exceptions:

- InvalidDateRangeException

- **Swagger Documentation (YAML):** yaml /api/v1/devices/{device_id}/history: get: summary: Get Historical Telemetry description: Retrieves data series for mobile graph visualization. security: - BearerAuth: [] parameters: - in: path name: device_id required: true schema: type: string - in: query name: start_time required: true schema: type: integer - in: query name: end_time required: true schema: type: integer - in: query name: resolution schema: type: string enum: [raw, hourly, daily] responses: '200': description: Time-series telemetry points.
-

3.7 List Active Alerts

- **Purpose:** Fetches active warnings or critical events associated with the device.

- **HTTP Method:** GET

- **Route:** /api/v1/devices/{device_id}/alerts

- **Authentication:** OAuth2 Bearer (JWT)

Headers:

- **Authorization:** Bearer <JWT>

Query Parameters:

- **resolved** (Boolean, optional. Default: false)

- **Request Body:** None

- **Response Body (200 OK - JSON):**

```
json [ { "alert_id": "alert-uuid-9999", "device_id": "ecobuck-dev-001", "alert_type": "moisture_extreme", "message": "Compost humidity is 15% (Too Dry). Please add water.", "resolved": false, "triggered_at": "2026-07-08T08:10:00Z" } ]
```

Error Codes:

- **403 Forbidden:** Access denied.

- **Validation Rules:** None.

- **Possible Exceptions:** None.

- **Swagger Documentation (YAML):**

```
yaml /api/v1/devices/{device_id}/alerts: get: summary: List Alerts description: Retrieves warnings like sensor dropouts, temperature peaks, or offline events. security: - BearerAuth: [] parameters: - in: path name: device_id required: true schema: type: string - in: query name: resolved schema: type: boolean responses: '200': description: Alert logs array.
```

3.8 Resolve Alert

- **Purpose:** Acknowledges or manually clears a warning state.

- **HTTP Method:** POST

- **Route:** /api/v1/devices/{device_id}/alerts/{alert_id}/resolve

- **Authentication:** OAuth2 Bearer (JWT)

Headers:

- **Authorization:** Bearer <JWT>

- **Request Body:** None
- **Response Body (200 OK - JSON):** `json { "alert_id": "alert-uuid-9999", "resolved": true, "resolved_at": "2026-07-08T08:25:00Z" }`

Error Codes:

- `403 Forbidden`: Access denied.
 - `404 Not Found`: Alert ID does not exist for this device.
- **Validation Rules:** Path arguments must be valid formats.

Possible Exceptions:

- `AlertAlreadyResolvedException`
- **Swagger Documentation (YAML):** `yaml`
`/api/v1/devices/{device_id}/alerts/{alert_id}/resolve: post: summary: Resolve Alert`
`description: Marks an active alert as resolved in the database. security: - BearerAuth:`
`[] parameters: - in: path name: device_id required: true schema: type: string - in: path`
`name: alert_id required: true schema: type: string responses: '200': description: Alert`
`successfully resolved. '404': description: Alert not found.`

3.9 Manual Compost Validation

- **Purpose:** Allows the user to confirm the compost is physically ready (dark brown, crumbly, sweet smell), triggering transition from `ready_candidate` to `ready_confirmed`.
- **HTTP Method:** `POST`
- **Route:** `/api/v1/devices/{device_id}/validate`
- **Authentication:** OAuth2 Bearer (JWT)

Headers:

- `Authorization: Bearer <JWT>`
 - `Content-Type: application/json`
- **Request Body (JSON):** `json { "user_notes": "Compost smells like fresh soil, ready to use on garden." }`
 - **Response Body (200 OK - JSON):** `json { "device_id": "ecobuck-dev-001", "old_status": "ready_candidate", "new_status": "ready_confirmed", "validated_at": "2026-07-08T08:25:30Z" }`

Error Codes:

- **400 Bad Request:** Compost state is not currently `ready_candidate` (cannot validate compost that hasn't completed decomposition).
- **403 Forbidden:** Access denied.

Validation Rules:

- `user_notes` is optional, max 500 characters.

Possible Exceptions:

- **InvalidCompostStateException:** Triggered if the device is not in `ready_candidate` status when user attempts validation.
- **Swagger Documentation (YAML):** `yaml /api/v1/devices/{device_id}/validate: post: summary: Validate Compost Maturity description: Triggers validation sequence, moving compost status to `ready_confirmed`. security: - BearerAuth: [] parameters: - in: path name: device_id required: true schema: type: string requestBody: required: false content: application/json: schema: type: object properties: user_notes: type: string responses: '200': description: State transitioned. '400': description: Compost not ready for validation.`

3.10 Update Device Configuration

- **Purpose:** Updates physical operational constraints (e.g., changing reading frequency, calibrating temperature offsets) for a device.
- **HTTP Method:** `PUT`
- **Route:** `/api/v1/devices/{device_id}/config`
- **Authentication:** OAuth2 Bearer (JWT)

Headers:

- `Authorization: Bearer <JWT>`
- `Content-Type: application/json`

- **Request Body (JSON):** `json { "temp_offset_c": -0.8, "humidity_offset_pct": 1.5, "read_interval_seconds": 60, "summary_interval_seconds": 600 }`
- **Response Body (200 OK - JSON):** `json { "device_id": "ecobuck-dev-001", "updated_config": { "temp_offset_c": -0.8, "humidity_offset_pct": 1.5, "read_interval_seconds": 60, "summary_interval_seconds": 600 }, "sync_pending": true }`

Error Codes:

- **400 Bad Request:** Attempting to set operational variables out of bounds (e.g., negative sleep intervals).
- **403 Forbidden:** Access denied.

Validation Rules:

- **read_interval_seconds** must be between 10 and 3600.
- **summary_interval_seconds** must be between 30 and 86400.

Possible Exceptions:

- **InvalidConfigurationParametersException**
- **Swagger Documentation (YAML):**

```
yaml /api/v1/devices/{device_id}/config: put: summary: Update Device Configuration description: Modifies device offset, polling rates, and sync timing parameters. security: - BearerAuth: [] parameters: - in: path name: device_id required: true schema: type: string requestBody: required: true content: application/json: schema: type: object properties: temp_offset_c: type: number humidity_offset_pct: type: number read_interval_seconds: type: integer summary_interval_seconds: type: integer responses: '200': description: Config updated in DB. '400': description: Parameters invalid.
```